PJ1

Overview

In this project you will implement an AVL tree and a red–black tree, compare their empirical performance, explain the difference using theoretical properties, and propose a possible improvement to a red–black tree. The two trees must expose the same ordered-set interface so that the experiment compares data structures rather than unrelated programs. You may use any programming language approved for the course. Except where a task explicitly says otherwise, the implementation, correctness argument, and performance analysis must be your own work.

Store one comparable value per node (for example, a signed 64-bit integer); the value itself determines the ordering. Values are unique: inserting an existing value does not create a second node. Unless otherwise stated, an unsuccessful search or deletion is a normal operation and must not crash.

1 Common interface and conventions

Provide an implementation for each tree with the following logical operations. Names may differ by language, but the meaning and return values must be documented in the README.

Operation	Required behavior
`insert(value)`	Insert the value; report whether a new value was inserted.
`find(value)`	Report whether the value is present.
`erase(value)`	Remove the value if present; report whether removal occurred.
`size()`	Return the number of stored values.
`empty()`	Return whether the tree contains no values.
`clear()`	Remove all values and release owned memory.
`inorder()`	Return or visit values in strictly increasing order.
`height()`	Return the height using one stated convention (empty tree must be specified).
`validate()`	Check ordering and all balance/color invariants; intended for testing.
`print_standard()`	Print the required level-order representation in Section 4.

The public interface must not expose implementation-specific balancing steps. A tree owns its nodes and must not leak memory. If you use a sentinel NIL node for the red–black tree, document whether it is counted in height or output (it must not be printed as a data node).

AI use: No. You may use AI only to clarify language syntax or tooling, not to design, implement, or explain these interfaces.

2 AVL tree

Implement an AVL tree supporting all operations in Section 1, including insertion and deletion rebalancing. Each real node must maintain enough metadata to compute its balance factor efficiently (normally a height or balance field). After every successful update, the AVL invariant must hold:

$$|h(\text{left}) - h(\text{right})| \leq 1$$

for every node. Implement and use the four rotation cases (LL, RR, LR, RL), or an equivalent decomposition into single rotations.

Your report must include pseudocode or a concise explanation for update rebalancing, a proof sketch of the invariant, and worst-case asymptotic bounds for search, insertion, deletion, and extra space. State the height convention used by your code.

AI use: No. AI may not write or rewrite AVL implementation, balancing logic, pseudocode, proof, or complexity analysis.

3 Red–black tree

Implement a red–black tree supporting the same interface. Use either a shared NIL sentinel or null children, but state the choice clearly. Every real node must have a color and the implementation must maintain the following invariants:

1. Every node is red or black; the root is black.
2. All NIL/null leaves are treated as black.
3. A red node has only black children.
4. Every path from a node to descendant NIL leaves contains the same number of black nodes.

Implement both insertion fix-up and deletion fix-up; a library ordered-set or tree routine may not replace your implementation. The report must explain the cases handled by each fix-up, give a proof sketch for preservation of the invariants, and state worst-case bounds for all required operations.

AI use: No. AI may not write or rewrite red–black tree implementation, fix-up cases, pseudocode, proof, or complexity analysis.

4 Standard tree output and testability

Both implementations must provide `print_standard()` (or a documented equivalent) with a deterministic, byte-for-byte comparable structure. Print the tree recursively using parentheses: each real node is written as `(value left right)`, where `left` and `right` are the parenthesized representations of its children. A missing child is written as `()`, and an empty tree is therefore `()`. For the AVL tree, print only the node value. For the red–black tree, print the node value followed by `:R` or `:B`; do not print height metadata. Separate tokens inside a node with single spaces and do not add trailing spaces. For example, one valid red–black output is:

```
(10:B (5:R () ()) (15:B (7:B () ()) ()))
```

If your language cannot preserve the literal format, provide a deterministic text file output with the exact same tokens and document the adaptation. Include automated tests for empty trees, duplicate insertion, unsuccessful operations, sorted and reverse-sorted insertion, all rotation/fix-up cases, and invariant checks after randomized mixed operations.

AI use: Yes, but only for the standard-output formatting, test-case design, and test code. The data-structure implementation and invariant checker logic must be written by you.

5 Performance experiment and theory

Use one common driver and identical operation sequences for both trees. Measure at least three input sizes spanning at least one order of magnitude (for example, 10^3, 10^4, and 10^5 values, subject to practical limits). Include at least the following workloads:

1. random unique-value insertion;
2. sorted insertion;
3. reverse-sorted insertion;
4. a mixed workload containing successful searches, unsuccessful searches, insertions, and deletions.

Warm up before timing, exclude input generation and printing from timed regions, repeat each measurement enough times to report a stable statistic, and state whether you report mean, median, or another statistic. Verify correctness and `validate()` after every workload. Report wall-clock time and, if available, comparison/rotation/recolor counters. Present a table or plot and explain sources of variation.

Relate the measurements to theory: compare the AVL height bound $O(\log n)$ with the red–black height bound at most $2\log_2(n+1)$; explain why AVL trees can have shorter search paths while red–black updates may require less rebalancing. Discuss workload dependence rather than claiming one structure is universally faster.

AI use: No for experiment design decisions, data collection, interpretation, theoretical explanation, and plots. AI-generated plotting/test boilerplate is allowed only under Section 4; all numbers must come from your own runs.

6 Red–black tree improvement proposal

Based on your measurements and literature/AI-assisted investigation, discuss one possible direction for improving a red–black tree or a related balanced binary search tree. The following are broad directions; you may choose one or develop another documented direction:

- memory representation, allocation, and locality;
- balance maintenance and update strategies;
- concurrency and parallel processing;
- workload-aware structures and operations.

State the main idea, its expected benefit, and its main trade-off. Distinguish clearly between an implemented result and a proposed direction, and cite relevant sources where appropriate.

AI use: Yes, but only for researching and brainstorming the improvement proposal, finding relevant references, and comparing possible directions. Any prototype code, measurements, and final technical judgment must be your own and clearly identified.

7 Required submission contents

Place all files in a directory named `pj1`. The directory must contain:

1. source code for both trees and the common driver;
2. automated tests and any small test data needed to run them;
3. `README` (or `README.md`) explaining prerequisites, compiler/interpreter version, build command, test command, experiment command, command-line arguments, input format, output format, and how to reproduce the reported measurements;
4. an experiment report in PDF. The report must include interface decisions, implementation and invariant explanations, correctness evidence, theoretical bounds, experimental methodology, raw or summarized data, tables/plots, discussion of performance differences, limitations, and the red–black improvement proposal;
5. a short AI-use disclosure listing which files or sections used allowed AI assistance and confirming that all disallowed implementation and analysis work is original.

The submitted code must compile/run from the documented commands without editing source files. Do not submit generated build directories, editor metadata, or external library source unless the course staff explicitly requests them.

Schedule. The total project and experiment period is three weeks; the submission deadline (DDL) is three weeks after the project is assigned.